



shiire-kanri 管理者向けマニュアル

Web アプリ運用・GAS 実装・デプロイ手順

最終更新: 2026年6月21日
shiire-kanri.nsdktts1030.workers.dev

目次

はじめに

システム全体像

未反映ゼロ保証アーキテクチャ

3 層構造

レイヤー 1: クライアント outbox(pages/app.js)

レイヤー 2: Workers 冪等性ミドルウェア(src/utils/idempotency.js)

レイヤー 3: 監視ポイント

新規外注スタッフの追加手順

ステップ 1: 作業者マスターに登録

ステップ 2: 5 分待つ

ステップ 3: 外注に URL を共有

ステップ 4: 外注本人がログイン

管理者の主な業務

1. AdminPanel(GAS エディタ内モジュール)

2. 請求書管理パネル

3. 月次運用ルーチン

4. AI 判定の運用

5. EC 連携

スプレッドシート構造

主要シート一覧(21+)

商品管理シート 主要列(STAFF_COL)

作業者マスターシート 列

請求書履歴シート 列構成

GAS 関数一覧

エントリポイント

認証・ユーティリティ

スタッフ向け API (action 一覧)

トリガーハンドラー

Workers ルート一覧

管理者・運用系

認証・マスター

商品・仕入れ

保存系

業務系

Push 通知

請求書(スタッフ)

請求書(管理者)

汎用

デプロイ手順

GAS デプロイ(shiire-kanri)

Cloudflare Workers デプロイ

Cron トリガー再登録

Workers Cron 確認

環境変数・シークレット一覧

GAS スクリプトプロパティ

Workers シークレット(wrangler secret)

Workers バインディング(wrangler.toml)

ビジネス設定 (AdminPanel から編集可能)

請求書フロー詳細

ステータス遷移

CSV / PDF 生成

CSV 28 項目

重複防止

トラブルシューティング

スタッフがログインできない

請求書 PDF がダウンロードできない

報酬集計が更新されない

Workers が応答しない

AI 判定が止まっている

EC 連携同期が遅延

スプレッドシート列を間違えて削除した

外注から「右下のバッジ(未送信)が消えない」と連絡が来た

同じ操作が二重登録された

仕入れ数(点数)を間違えて報告された

改良時のマニュアル更新フロー

参考リンク

付録: 各シートの初期化

アップデート履歴

はじめに

このマニュアルは、**shiire-kanri Web アプリ**の **管理者(オーナー)向け** の運用・保守手引きです。

外注向けの操作方法は別冊「外注向けマニュアル」に分離してあります。本書は管理者しか触らない領域(スプレッドシート構造・GAS 関数一覧・デプロイ手順・トラブルシューティング)まで網羅しています。

アプリURL(本番):

- フロント: <https://shiire-kanri.nsdktts1030.workers.dev/>
- GAS Web App: Cloudflare Workers の `GAS_API_URL` シークレットに格納

主要リポジトリ:

- GAS コード: `/Users/katsu/saisun-repo/shiire-kanri/`
- Workers コード: `/Users/katsu/saisun-repo/workers/shiire-kanri/`

システム全体像

```
ユーザー (外注スタッフ・管理者)
↓
Cloudflare Access (Email OTP 認証)
↓
Cloudflare Workers (shiire-kanri.nsdktts1030.workers.dev)
├ SPA 配信 (HTML/JS/CSS)
├ JWT 検証
├ /api/* ルーティング
│   ├── D1 / KV / R2 で高速処理
│   └─ 未対応 API は GAS にプロキシ
└ 5 分 Cron で GAS から作業員メール一覧を取得 → CF Access ポリシー同期
↓ (X-Sync-Secret 認証)
GAS Web App (仕入れ管理 GAS プロジェクト)
├ doPost: action ベース dispatch
└ doGet: SPA / AdminInvoice モーダル
↓
Google Sheets (DBの代替・スプレッドシート 1 個)
├ 商品管理 / 仕入れ管理 / 作業員マスター
├ 経費申請 / 仕入れ数報告 / 移動報告 / 返送管理
├ 報酬管理 / EC管理 / 売却履歴
├ 請求書履歴 / 請求書修正申請 / インボイス経過措置率 / 請求書管理者設定
├ AI画像判定 / AIキーワード抽出
└ 各種マスタ・設定・分析シート
```

役割分担:

- **Cloudflare Workers**: 認証・JSON 整形・キャッシュ・静的配信
- **GAS**: 業務ロジック・Sheets 操作・メール送信・PDF 生成
- **Sheets**: 永続データ
- **CF Access**: メール OTP 認証(実装は Cloudflare 側、コード変更不要)

未反映ゼロ保証アーキテクチャ

目的: 外注スタッフがアプリで実行した操作(採寸保存・写真アップ・経費申請・移動・請求書修正など)が **絶対にスプレッドシートへ反映漏れない** ことを保証する。圏外・5xx・タブ切断・ブラウザ強制終了などの障害をすべて吸収する。

3 層構造

```
[クライアント側 outbox] → IndexedDB (sk-outbox.queue)
  ↓ 復帰時に自動 flush (X-Idempotency-Key 付き)
[Workers 側 冪等性ミドルウェア] → D1: idempotency_keys (TTL 7日)
  ↓ 初回のみハンドラ実行
[GAS / Sheets]
```

レイヤー 1: クライアント outbox(pages/app.js)

- `api(path, { outbox: true, label })` で書き込み API を呼ぶと、IndexedDB `sk-outbox.queue` に **送信前** にレコードを積む
- ネットワーク失敗 / 5xx は「**予約しました**」トーストで成功扱い、4xx は破棄
- 復帰イベント(DOMContentLoaded・online イベント)で `flushOutbox_()` が走り、最大 30 回までリトライ
- 各リクエストにブラウザ生成の UUIDv4 を `X-Idempotency-Key` ヘッダで付与
- 右下に「🕒 **未送信 N件**」バッジ(`refreshOutboxBadge_`)。タップで詳細モーダル(`showOutboxModal_`)

対応済み API(14系統): 経費送信 / 移動 CRUD / 返品 CRUD / 作業者保存 / 仕入れ数量 / 請求書プロフィール / 請求書修正申請 / 同梱トグル など。画像アップロードは別系統(`outboxAdd_({type:'image'})`)。

レイヤー 2: Workers 冪等性ミドルウェア(src/utils/idempotency.js)

- D1 テーブル `idempotency_keys` (migration 004)
 - `key` PRIMARY KEY, `response_body` , `status_code` , `path` , `user_email` , `created_at` , `expires_at`
 - TTL 7 日(外注が長期オフラインでも安全に flush できる期間)
- `withIdempotency(request, env, handler)` ですべての write ルートをラップ(21箇所、`src/index.js`)
- 同じ `X-Idempotency-Key` で再送されると **D1 から 2xx レスポンスを再生** してハンドラを実行しない → GAS/Sheets への二重書き込みを防止
- 4xx/5xx はキャッシュしない(リトライ可能)

レイヤー 3: 監視ポイント

- D1 `idempotency_keys` の行数が異常に膨らんでいないか(通常運用なら数百件以内)

```
wrangler d1 execute shiire-kanri-db --remote \
  --command "SELECT COUNT(*) FROM idempotency_keys WHERE expires_at > strftime('%s','now')"
```

- 外注から「右下のバッジが消えない」と連絡が来たら、まずは本人のオンライン環境を確認。それでもダメな場合は Workers/GAS のエラーログを確認

新規外注スタッフの追加手順

外注を追加する基本フローは下記の 4 ステップ。

ステップ 1: 作業者マスターに登録

仕入れ管理スプレッドシートの「作業者マスター」シートを開く。

列	入力内容
B 列	氏名(一意の文字列。主キーとして使用)
C 列	フリガナ/別名(任意)
D 列	Gmail アドレス(メイン)
E 列	Gmail アドレス(サブ・任意)
O 列	チェックボックス(TRUE にする)

D 列または E 列に外注の Gmail を入力し、O 列にチェック。



B 列の氏名は他のスタッフと重複させないでください。同姓同名の場合は屋号付き・年齢付き等で区別。

ステップ 2: 5 分待つ

Cloudflare Workers の Cron トリガーが **5 分ごと** に GAS から許可メール一覧を取得し、CF Access ポリシーを更新します。

実装場所:

- 同期スクリプト: `workers/shiire-kanri/src/sync/access-sync.js`
- Cron 設定: `workers/shiire-kanri/wrangler.toml` (`*/5 * * * *`)

手動で即時反映したい場合は `POST /admin/sync-access` を叩く。

```
curl -X POST https://shiire-kanri.nsdktts1030.workers.dev/admin/sync-access \
-H "X-Admin-Key: <ADMIN_KEY>"
```

ステップ 3: 外注に URL を共有

```
https://shiire-kanri.nsdktts1030.workers.dev/
```

をメッセージや LINE で送る。

ステップ 4: 外注本人がログイン

- URL を開く
- Gmail アドレスを入力 → 「Send me a code」
- 受信した 6 桁コードを入力 → 「Sign in」
- アプリ画面が開く



外注側でやることは「URL を開いてメール入力 → コード入力」だけ。Google アカウント連携などは不要。

管理者の主な業務

1. AdminPanel(GAS エディタ内モジュール)

GAS エディタを開いて「★ 管理パネルを開く」メニューから起動。

実装: `shiire-kanri/AdminPanel.html` + `shiire-kanri/AdminPanelApi.gs`

主な機能:

機能	関数	ファイル:行
プロパティ表示・編集	<code>adminPanel_getProperties</code> / <code>adminPanel_setProperties</code>	AdminPanelApi.gs:16-45
新月処理	<code>startNewMonth</code>	AdminPanelApi.gs:51
当月管理番号同期	<code>syncCurrentMonthIds</code>	AdminPanelApi.gs:62
当月理論在庫再計算	<code>recalcCurrentTheory</code>	AdminPanelApi.gs:71
在庫推移更新	<code>updateInventoryTrend</code>	AdminPanelApi.gs:75
月次レポート生成	<code>generateReport</code>	AdminPanelApi.gs:79
EC連携同期	<code>ecSync</code>	AdminPanelApi.gs:83
返送ステータス同期	<code>returnStatusSync</code>	AdminPanelApi.gs:87
場所移動処理	<code>moveProcess</code>	AdminPanelApi.gs:91
仕入れ数マージ	<code>mergeShiire</code>	AdminPanelApi.gs:95
欠番チェック	<code>checkMissingIds</code>	AdminPanelApi.gs:99
入替リスト生成	<code>generateSwapLists</code>	AdminPanelApi.gs:103
AI 判定ステータス	<code>aiKeywordStatus</code>	AdminPanelApi.gs:112-169
トリガー管理	<code>listTriggers</code> / <code>rebuildTriggers</code> / <code>deleteTrigger</code>	AdminPanelApi.gs:175-199
ビジネス設定	<code>getBizSettings</code> / <code>setBizSettings</code>	AdminPanelApi.gs:205-242
メール設定	<code>getMailSettings</code> / <code>setMailSettings</code>	AdminPanelApi.gs:248-276



プロパティ画面は SECRET/TOKEN/PASSWORD/KEY パターンが自動マスクされます。表示には「マスクを解除」操作が必要です。

2. 請求書管理パネル

GAS エディタ → 「📄 請求書管理を開く」メニュー、または Web 画面の管理者メニューから。

実装: `shiire-kanri/AdminInvoice.html` + `shiire-kanri/AdminInvoiceApi.gs` + Workers `/api/admin-invoice/*`

タブ構成:

全請求書タブ

全スタッフの全請求書を一覧表示。月・ステータス・スタッフ名でフィルタ可能。

ステータス変更(未確認 → 確認済み → 修正申請中 → 修正承認済み → 作成済み → 支払済み → 取消済み)はここから操作。

「**↓ 振込用まとめCSV**」ボタン: 現在の一覧(フィルタ結果)を、請求月・請求書番号・スタッフ名・本名・銀行名・支店名・口座種別・口座番号・口座名義・振込先希望銀行・税込合計・調整額・振込手数料・請求額・ステータスの列で CSV 出力します(Excel 文字化け防止の BOM 付き)。月末の一括振込作業に使用します。

「**詳細**」ボタン → **請求書詳細モダル**:

- 請求書の全項目(プロフィール・件数・金額・振込先)を確認できます。
- 追加報酬・控除(手動明細)の編集**: 管理者は、任意スタッフの請求書に管理者レイヤーの自由明細(名目+金額・複数行・±可)を追加・編集できます。スタッフが自分で追加した行は **読み取り専用** で表示され、管理者は触れません。金額左の **+/- ボタン** でプラス/マイナスを切替。「保存」で管理者レイヤーのみ更新(自動算出値・スタッフ追加分は不変)。
- 「自動算出を再計算」ボタン**: 件数・報酬の自動算出値が誤っている場合に使用。最新のソースデータ(報酬管理/商品管理/管理者設定)で計算し直し、**現在の請求書を取消にして新しい請求書番号で再発行** します(手動明細は引き継がれます)。誤った金額を手で書き換えるのではなく、元データを直してからこのボタンで作り直すのが正しい運用です。
- 「支払済み」「取消済み」の請求書は編集できません(手動明細・再計算ボタンは非表示になり、内容は読み取り専用で表示)。

修正申請タブ

スタッフからの修正申請を一覧表示。承認・却下・差戻しのいずれかを選択し、コメントを添えて送信。スタッフにメール通知。

経過措置率タブ

インボイス制度の経過措置率を区間ごとに管理。

区間	控除可能率	デフォルト終了日
～2026/09	80%	2026/09/30
2026/10～2029/09	50%	2029/09/30
2029/10～	0%	永続

開始/終了 YM・控除可能率を編集すると、新規請求書プレビューに即反映されます。

管理者設定タブ

請求書の発行元情報を管理。

項目	内容
有効	TRUE で請求書発行を許可
屋号	受領者として表示
インボイス番号	自社の登録番号
振込元銀行候補	スタッフが選択できる銀行リスト
振込手数料(楽天⇄楽天)	0 円固定
振込手数料(他行小額)	例: 165 円
振込手数料(他行高額)	例: 270 円
通知先メール	修正申請発生時の通知先

3. 月次運用ルーチン

月末(毎月末日)

- `updateRewardsNoFormula(false)` 実行(自動 Cron) → 報酬管理シート更新
- `generateMonthlyReport` 実行(自動 Cron) → 月次レポート生成
- 仕入れ・販売データの締め

月初(1～5日)

- 前月の売上・利益を確認
- スタッフからの修正申請対応

月中(20日～25日頃)

- スタッフが前月分の請求書を作成・提出
- 修正申請が来たら速やかに対応(メールで通知が届く)
- 承認後、銀行振込

振込後

- 該当請求書のステータスを「支払済み」に変更

4. AI 判定の運用

タスキ箱(画像アップロードツール)から商品画像がアップされると、5 分 Cron で Gemini API による判定が走り、AI画像判定シートに結果が書き込まれます。

詳細: `project-tasukibako-ai-integration` 関連メモ。

- 1 分 Cron: `processPendingKeywordRows` (AIキーワード抽出.gs)
- 5 分 Cron: AppSheet/Gemini 判定(gas-proxy 側)

5. EC 連携

BASE で売れた注文は 5 分 Cron で「依頼管理」シート→「EC管理」シートに自動反映されます。

- 関数: `syncBaseOrdersToEc` (EC管理自動反映.gs:44)
- 関連プロパティ: `EC_SYNC_SRC_SPREADSHEET_ID` , `EC_SYNC_DST_SPREADSHEET_ID`

スプレッドシート構造

仕入れ管理スプレッドシートは **1 個のスプレッドシート** に複数シートが入っています。スプレッドシート ID はスクリプトプロパティ `SPREADSHEET_ID` に格納。

主要シート一覧(21+)

シート名	用途	列数	主要列マップ参照
商品管理	商品マスター	68	StaffApi.gs:4-42 STAFF_COL
仕入れ管理	購買記録	多 列	仕入れ管理.gs
作業者マスター	従業員管理	41 +	StaffApiExtras.gs:405-432 / InvoiceSheetSetup.gs:14-28
請求書履歴	請求書履歴	45	InvoiceSheetSetup.gs:46-56
請求書修正申請	修正申請管理	13	InvoiceSheetSetup.gs:65-69
インボイス経過措置率	税務経過措置	5	InvoiceSheetSetup.gs:76-86
請求書管理者設定	管理者設定	14	InvoiceSheetSetup.gs:92-96
移動報告	場所移動記録	6	StaffApiExtras.gs:16-49
返送管理	返送記録	7	StaffApiExtras.gs:91-123
経費申請	経費申請	10 +	StaffApiExtras.gs:709-732
仕入れ数報告	仕入数報告	7+	StaffApiExtras.gs:761-764
AIキーワード抽出	AI 抽出結果	10 +	キーワードAPI.gs:42-58
AI画像判定	AI 判定結果	多 列	AdminPanelApi.gs:115-157
EC管理	EC売上管理	12 +	EC管理自動反映.gs:76-128
依頼管理	saisun-list 側注 文	多 列	(別プロジェクト連動)
売却履歴	売上記録	多 列	報酬更新.gs
回収完了	在庫回収	多 列	メニュー.gs:39-46
納品場所	納品先マスター	多 列	場所.gs
管理番号マスタ	管理番号採番	3+	番号採番.gs
仕入先マスタ	仕入先マスター	多 列	仕入先.gs
設定	汎用設定	多 列	Config.gs
作業分析 / 在庫分析 / 月次推 移	分析レポート	多 列	各レポート.gs

商品管理シート 主要列(STAFF_COL)

shiire-kanri/StaffApi.gs:4-42 に定義。

列	フィールド	用途
A (1)	管理番号	zkXXXX 主キー
B (2)	仕入れID	仕入れ管理シートとの紐付け
C (3)	作業者名	作業担当者
D-T (4-20)	採寸寸法 12 項目	着丈・肩幅・身幅 等
U (21)	採寸日	タイムスタンプ
V (22)	採寸者	名前
W-AB (23-28)	商品名・カテゴリ・ブランド・色・素材・状態	
AJ (42)	販売日	
AK (43)	販売場所	メルカリ / BASE / ジモティ等
AL (44)	販売価格	
AM (45)	送料	
AN (46)	手数料	
AS (65)	販売タイムスタンプ	onEdit で自動



列番号固定参照は将来の列追加で壊れます。新規実装ではヘッダー名動的解決 (`Utils.gs:13-40` の `buildHeaderMap_` / `findColByName_` / `requireCol_`) を使ってください。

作業者マスターシート 列

列	内容	用途
B	氏名	主キー
D / E	Gmail	CF Access 認証
F-M	単価マスター	採寸／撮影／出品／発送 等の単価
O	有効フラグ	TRUE でアプリログイン可
Q (17)	管理者フラグ	TRUE で管理者権限
AC-AO (29-41)	請求書拡張	屋号／本名／住所／口座／インボイス番号

請求書拡張列のマップ: `InvoiceSheetSetup.gs:14-28` の `INV_WORKER_EXT_COL`

請求書履歴シート 列構成

`InvoiceSheetSetup.gs:46-56` の `INV_HISTORY_HEADERS` に 45 列分のヘッダー定義。

主要列:

- A: 請求書番号 `INV-YYYYMM-{row}-{seq}`
- B: 請求月 `YYYY/MM`
- C: スタッフ名
- E-P: スナップショット情報(屋号・本名・住所・口座等)
- Q-T: 件数(採寸／撮影／出品／発送)

- U-X: 単価
- AD: 税込合計
- AE: 控除率
- AF: 調整額
- AG: 振込元
- AH: 手数料
- AI: 請求額
- AJ: ステータス(7段階)
- AN: スナップショットJSON(全データの不変コピー)

GAS 関数一覧

`/Users/katsu/saisun-repo/shiire-kanri/` 配下の主要 .gs ファイル。

エントリポイント

関数	ファイル:行	用途
<code>doPost(e)</code>	Code.gs:5-104	Web API メイン dispatcher。X-Sync-Secret 認証+ action ベース分岐
<code>doGet(e)</code>	Code.gs:78-90	SPA・AdminInvoice モーダル配信
<code>onOpen()</code>	メニュー.gs:16-21	スプレッドシート開封時にメニュー登録

認証・ユーティリティ

関数	ファイル:行	用途
<code>staff_isWhitelisted_</code>	StaffApi.gs:380	メールアドレスが許可リストにあるか判定
<code>staff_assertSagyouAdmin_</code>	StaffApiExtras.gs:597	管理者フラグの有無を検査
<code>inv_resolveStaffByEmail_</code>	InvoiceApi.gs:131	メールから作業者マスターの行を解決
<code>adminInv_assertAdmin_</code>	AdminInvoiceApi.gs:16	管理者専用ガード
<code>buildHeaderMap_</code>	Utils.gs:13	1 行目をヘッダー名→列インデックスのマッピング化
<code>findColByName_</code>	Utils.gs:25	ヘッダー名で列番号取得
<code>requireCol_</code>	Utils.gs:35	必須ヘッダーが無ければエラー
<code>withLock_</code>	Utils.gs	DocumentLock で排他処理
<code>cleanupStaleProps</code>	Utils.gs:119-157	古いスクリプトプロパティ削除(毎日 5 時 Cron)

スタッフ向け API (action 一覧)

`doPost` の switch から分岐される action 群(X-Sync-Secret 認証必須)。

データ参照系:

- `syncDumpProducts` / `syncDumpPurchases` / `syncDumpAiPrefill` — Workers KV 用ダンプ
- `dumpHeaders` / `dumpSheet` — 汎用ダンプ
- `listAllowedEmails` — CF Access 同期用
- `listWorkers` / `listAccounts` / `listSuppliers` / `listPlaces` / `listCategories` / `listSettings` — マスター取得

- `lookupAiPrefill` — AI 判定結果検索

商品 CRUD:

- `saveMeasurement` — 採寸保存
- `saveSale` — 販売情報保存
- `saveDetails` — 商品詳細保存
- `createProduct` / `deleteProduct` — 商品作成・削除
- `uploadImage` — 画像アップロード(Drive)

業務 API:

- `listMoves` / `createMove` / `updateMove` / `deleteMove` — 場所移動
- `listReturns` / `createReturn` / `updateReturn` / `deleteReturn` — 返送管理
- `listAiResults` — AI 判定結果一覧
- `listSagyousha` / `saveSagyousha` / `createSagyousha` — 作業者管理
- `appendKeihi` / `uploadKeihiImage` — 経費申請
- `updateShiireHoukokuQuantity` — 仕入れ数報告

請求書 API(スタッフ):

- `invoiceCurrentUser` — 自身の情報取得
- `listInvoices` — 自身の請求書一覧
- `getInvoiceDetail` — 詳細
- `listMyAvailableMonths` — 利用可能月
- `calcInvoicePreview` — 見積計算
- `getInvoiceProfile` / `saveInvoiceProfile` — プロフィール
- `createInvoice` — 請求書作成
- `downloadInvoicePdf` — PDF ダウンロード
- `requestInvoiceRevision` — 修正申請
- `listMyRevisions` — 自身の修正申請一覧

請求書 API(管理者):

- `adminInv_listAllInvoices` — 全請求書
- `adminInv_getInvoiceDetail` — 詳細
- `adminInv_listAllRevisions` — 全修正申請
- `adminInv_respondRevision` — 修正申請対応
- `adminInv_updateInvoiceStatus` — ステータス変更
- `adminInv_listGraceRates` / `adminInv_saveGraceRates` — 経過措置率
- `adminInv_getAdminSettings` / `adminInv_saveAdminSettings` — 管理者設定

トリガーハンドラー

`shiire-kanri/トリガー設定.gs:2-30` に登録。

ハンドラー	起動条件	用途
stampByThreshold	毎日 0:30 JST	在庫日数スタンプ
handleChange_Mailer	onChange	依頼管理シート編集でメール通知
handleChange_Inventory	onChange	在庫同期
handleChange_Move	onChange	場所移動報告→商品管理に反映
handleChange_Return	onChange	返送管理→ステータス更新
handleChange_ShiireSync	onChange	仕入れ数報告→仕入れ管理マージ
processPendingKeywordRows	1 分ごと	AI キーワード抽出
buildWorkAnalysis	1 時間ごと	作業分析レポート
cronDaily3	毎日 3:00 JST	syncRewardRows + updateRewardsNoFormula + 欠番確認
recalcZaikoNissu	毎日 4:00 JST	在庫日数再計算
cleanupStaleProps	毎日 5:00 JST	プロパティクリーンアップ
syncBase0OrdersToEc	5 分ごと	BASE → EC管理シート同期

FULL_RESTORE_ALL() (shiire-kanri/トリガー設定.gs:2) でトリガー一括登録。

Workers ルート一覧

/Users/katsu/saisun-repo/workers/shiire-kanri/src/index.js で定義。全 60+ エンドポイント。

管理者・運用系

メソッド	パス	関数 / ファイル:行
GET	/health	index.js:38-40
POST	/admin/sync	手動同期 (index.js:47)
POST	/admin/sync-access	CF Access ポリシー手動同期 (index.js:61)
POST	/admin/backfill-thumb-url	thumb_url 一括バックフィル (index.js:69)

認証・マスター

メソッド	パス
GET	/api/me
GET	/api/master/workers / /accounts / /suppliers / /places / /categories / /settings

商品・仕入れ

メソッド	パス
GET/DELETE	/api/products , /api/products/{kanri} , /api/products/{kanri}/images
GET	/api/products/counts / /api/products/has-images
POST	/api/products/thumbs
GET	/api/kanri/next
GET/DELETE	/api/purchases , /api/purchases/{shiireId} , /api/purchases/{shiireId}/products

保存系

メソッド	パス
POST	/api/save/measurement / /sale / /details / /image
POST	/api/image/resolve
GET	/api/img / /api/thumb
POST	/api/create/purchase / /api/create/product

業務系

メソッド	パス
GET/POST/PUT/DELETE	/api/moves , /api/moves/{moveId}
GET/POST/PUT/DELETE	/api/returns , /api/returns/{boxId}
GET	/api/ai/list , /api/ai/prefill
POST	/api/ai/prefill/batch
GET/POST	/api/sagyousha , /api/sagyousha/create
GET/POST	/api/bundles , /api/bundles/toggle
GET	/api/sales/summary
POST	/api/keihi/submit , /api/keihi/image
POST	/api/shiire-houkoku/quantity
GET	/api/listing-text/{kanri}

Push 通知

メソッド	パス
GET	/api/push/vapid
POST	/api/push/subscribe / /unsubscribe / /test
GET/POST	/api/push/prefs

請求書(スタッフ)

メソッド	パス
GET	/api/invoice/me / /list / /detail / /months
POST	/api/invoice/preview
GET/POST	/api/invoice/profile
POST	/api/invoice/create
GET	/api/invoice/pdf
POST	/api/invoice/revision
GET	/api/invoice/revisions

請求書(管理者)

メソッド	パス
GET	/api/admin-invoice/list / /revisions
POST	/api/admin-invoice/revisions / /status
GET/POST	/api/admin-invoice/grace-rates / /settings

汎用

メソッド	パス
GET	/api/sheet/{name} — シート汎用ダンプ(業務メニュー)
GET/その他	/path — 静的アセット(SPA fallback)

デプロイ手順

GAS デプロイ(shiire-kanri)

```
cd /Users/katsu/saisun-repo/shiire-kanri

clasp push 2>&1 && \
DEPLOY_ID="<shiire-kanri 用 DEPLOY_ID>" && \
clasp deploy -i "$DEPLOY_ID" --description "変更内容" 2>&1
```



`clasp push` だけでは本番 Web App に反映されません。必ず `clasp deploy -i "$DEPLOY_ID"` をセットで実行してください。



DEPLOY_ID を間違えると別 URL の新デプロイが作成されます。本番は 1 本に固定。

Cloudflare Workers デプロイ

```
cd /Users/katsu/saisun-repo/workers/shiire-kanri

# 本番
wrangler deploy

# 開発環境
wrangler deploy --env dev
```

Workers の本番 URL: <https://shiire-kanri.nsdkttts1030.workers.dev/>

Cron トリガー再登録

GAS 側のトリガーが消えた場合は、GAS エディタから `FULL_RESTORE_ALL()` を 1 回実行すれば全トリガー再登録できます。

```
FULL_RESTORE_ALL() // トリガー設定.gs:2
```

Workers Cron 確認

```
cd /Users/katsu/saisun-repo/workers/shiire-kanri
wrangler tail # ライブログ
```

`wrangler.toml` の `[triggers]` で `crons = ["*/5 * * * *"]` を確認。

環境変数・シークレット一覧

GAS スクリプトプロパティ

`Utils.gs:106-112` の `KEEP_PROPS_` リストで cleanup から保護されているキー:

キー	用途
<code>SHIIRE_SYNC_SECRET</code>	Workers ↔ GAS 認証トークン
<code>OPENAI_API_KEY</code>	Gemini / ChatGPT API キー
<code>SPREADSHEET_ID</code>	メインスプレッドシート ID
<code>IMAGE_FOLDER_ID</code>	Drive 画像保存フォルダ ID
<code>INV_BUSY</code>	請求書生成ロック
<code>SWAP_EMAIL_FURUGIYAHONPO</code>	入替リスト通知先 1
<code>SWAP_EMAIL_HOSHIIGA</code>	入替リスト通知先 2
<code>EC_SYNC_SRC_SPREADSHEET_ID</code>	EC同期ソースID
<code>EC_SYNC_DST_SPREADSHEET_ID</code>	EC同期先ID
<code>XLSX_SOURCE_SPREADSHEET_ID</code>	XLSX ダウンロードソース
<code>XLSX_REQUEST_SPREADSHEET_ID</code>	XLSX リクエストシート
<code>OWNER_USER_KEYS</code>	所有者キー

Workers シークレット(wrangler secret)

```
cd /Users/katsu/saisun-repo/workers/shiire-kanri
```

```
wrangler secret put GAS_API_URL          # GAS Web App URL
wrangler secret put SYNC_SECRET          # GAS の SHIIRE_SYNC_SECRET と一致させる
wrangler secret put CF_ACCESS_TEAM       # Cloudflare Account ID
wrangler secret put CF_ACCESS_AUD        # CF Access Application ID
wrangler secret put ADMIN_KEY            # 管理者エンドポイント認証
wrangler secret put CF_API_TOKEN         # CF API (Access ポリシー更新用)
```

Workers バインディング(wrangler.toml)

種別	名前	用途
D1	(databases)	商品インデックス・キャッシュ
KV	CACHE	短期キャッシュ
KV	GAS_PROXY_CACHE	GAS レスポンスキャッシュ
R2	detauri-images	画像ストレージ
Assets	ASSETS	SPA 静的ファイル

ビジネス設定(AdminPanel から編集可能)

AdminPanelApi.gs:205-242 の CONFIG_BIZ_SETTINGS :

キー	内容
baseFeeRate / baseFeeFix	BASE 手数料率／固定額
creditRate	クレカ決済手数料率
paypayRate	PayPay 手数料率
konbiniRate	コンビニ決済手数料率
bankRate	銀行振込手数料率
payeasyRate	Pay-easy 手数料率
paidyRate	Paidy 手数料率
jimotiRate	ジモティ手数料率
aiModel	Gemini モデル名
aiDailyLimit	AI 判定の 1 日上限
aiMaxKeywords / aiMinKeywords	AI キーワード抽出範囲
rewardStartYear / rewardStartMonth	報酬集計の開始年月
swapTriggerDay	入替リスト生成日
monthlyReportFeeRate	月次レポート手数料率

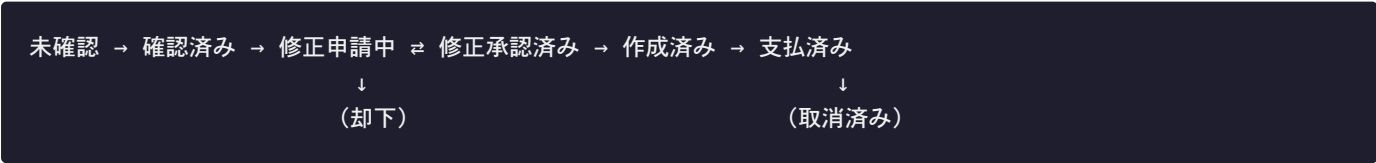
メール設定 (`AdminPanelApi.gs:248-276`):

キー	内容
<code>settingsSheet</code>	通知先メールアドレスシート名
<code>recipientCol</code> / <code>recipientStartRow</code>	宛先列・開始行
<code>shiireSubject</code>	仕入れ通知件名フォーマット
<code>expenseSubject</code>	経費通知件名フォーマット
<code>swapSubjectFormat</code>	入替リスト件名フォーマット

請求書フロー詳細

ステータス遷移

7 段階のステータスがあり、`InvoiceSheetSetup.gs:59` の `INV_STATUS_LIST` で定義。



CSV / PDF 生成

- 文字コード: UTF-8 BOM 付き
- 改行: CRLF
- ファイル名: `{本名}_{請求額}円_{YYYY年MM月}.csv` (/ は - に正規化)
- PDF: GAS で生成し base64 エンコード → Workers が `application/pdf` で返却 (`workers/shiire-kanri/src/handlers/invoice.js:75-94`)

CSV 28 項目

請求書番号 / 請求月 / 屋号 / 本名 / 郵便 / 住所 / 電話 / メール / インボイス番号 / 銀行名 / 支店 / 口座番号 / 採寸件数 / 撮影件数 / 出品件数 / 発送件数 / 在庫管理報酬 / 固定報酬 / 経費合計 / 売上報酬 / その他報酬 / 税込合計 / インボイス控除率 / 調整額 / 振込元 / 手数料 / 請求額 / 作成日時

重複防止

- `LockService.getDocumentLock().tryLock(20000)` で同時押下競合を防止
- 同月再作成時は既存レコードを返す
- 「取消」後の再作成は連番 `-2` , `-3` , ... が振られる

トラブルシューティング

スタッフがログインできない

- 作業者マスター D 列または E 列に Gmail が入っているか確認
- O 列(有効フラグ)が TRUE か確認
- 5 分待っても入れない場合は手動同期:

```
curl -X POST https://shiire-kanri.nsdkttts1030.workers.dev/admin/sync-access \
-H "X-Admin-Key: <ADMIN_KEY>"
```

4. Cloudflare ダッシュボード → Access → Applications → shiire-kanri → Policy で該当メールが Include 側にあるか確認

請求書 PDF がダウンロードできない

1. `INV_BUSY` プロパティが残っていないか確認(残っていれば削除)
2. PDF 生成は GAS 側で Drive 上の Google Doc を経由するため、Drive 容量超過していないか確認
3. スタッフのプロフィールが全項目埋まっているか確認

報酬集計が更新されない

1. 毎日 3 時の Cron (`cronDaily3`) が動いているか:
 - GAS エディタ → トリガー画面で実行履歴を確認
 - エラーがあれば `FULL_RESTORE_ALL()` で再登録
2. 手動実行: `updateRewardsNoFormula(false)` を直接呼ぶ

Workers が応答しない

1. `wrangler tail` でログ確認
2. CF Access ポリシーで自分自身が締め出されていないか確認
3. Cloudflare ダッシュボードで Workers のデプロイ状態を確認

AI 判定が止まっている

1. `processPendingKeywordRows` の Cron が動いているか
2. `OPENAI_API_KEY` が有効か
3. AdminPanel の「AI 判定ステータス」で今日の使用量を確認
4. `aiDailyLimit` に達していないか確認

EC 連携同期が遅延

1. `syncBaseOrdersToEc` (EC管理自動反映.gs:44) の Cron が動いているか
2. `EC_SYNC_SRC_SPREADSHEET_ID` / `EC_SYNC_DST_SPREADSHEET_ID` が正しいか
3. 手動実行で同期可能

スプレッドシート列を間違えて削除した

1. スプレッドシート → ファイル → 版の履歴 → 版の履歴を表示
2. 影響を受けた版より前の版に戻す
3. 戻せない場合は手動で列を再追加し、`buildHeaderMap_` のヘッダー名と一致させる

外注から「右下のバッジ(未送信)が消えない」と連絡が来た

1. 本人のオンライン環境を確認 — モバイルデータ ON / Wi-Fi 接続 / 機内モード OFF
2. ブラウザを開き直して `flushOutbox_` の自動再送を待つ(1~2分)
3. それでも消えない場合は Workers ログを確認

```
cd /Users/katsu/saisun-repo/workers/shiire-kanri
wrangler tail
```

4. 4xx ループ(バリデーションエラー)の場合は外注に「破棄」操作を案内
5. D1 障害が疑われる場合は冪等性テーブルを確認

```
wrangler d1 execute shiire-kanri-db --remote \
--command "SELECT COUNT(*) FROM idempotency_keys"
```

同じ操作が二重登録された

- 通常起きないはず (Workers の `withIdempotency` でブロック)
- もし発生したら、外注がブラウザのキャッシュクリア等で **IndexedDB を消した直後** に同じ操作を再実行した可能性
- スプレッドシート側で重複行を 1 件削除すれば復旧

仕入れ数(点数)を間違えて報告された

外注から「数を間違えて報告した」と連絡が来たときの対応です。外注向けマニュアルでは「自分で直さず管理者へ連絡」と案内しています(仕入れ数報告アプリは一度報告した行を編集できない仕様 — `staff_apiUpdateShiireHoukokuQuantity` が「既に処理済み」で拒否)。

連絡を受けたら、**LINE などで「区分コード」と「正しい総点数」**を確認してください。修正はスプレッドシートの専用メニューで自動化されています。

なぜ慎重に直す必要があるか

割り当て管理番号は区分コードごとに **点数を累計した連番** です。報告済みの点数をあとから変えて再採番させると、**同じ区分の後続バッチの管理番号が丸ごとズレ**、すでに採寸・撮影・出品まで終わった商品の番号が合わなくなります(管理番号はタスキ箱の画像キーでもあるため、ズレると画像も連動して壊れます)。そのため点数修正は専用メニューで安全に行います。

修正手順:「 仕入れ点数を修正」メニュー

1. **仕入れ管理シート**を開き、修正したい仕入れ行のセルをどれか 1 つ選択する
2. メニューバーの「**管理メニュー**」→「 **仕入れ点数を修正**」をクリック
3. 対象 ID・区分・現在の点数が表示されるので、**正しい総点数**を入力して **OK**
4. 修正内容のプレビューが出るので、確認して **はい**

これだけで、後述の処理(補助行作成・金額按分・原価再同期・管理番号採番)がすべて自動で実行されます。**仕入れ点数修正.gs** 参照。

点数を増やすとき(過少報告:例 50→52)

- 差分(例 2 点)の **補助行を自動作成**し、区分の **最後尾** に管理番号を採番します(既存の番号は 1 つもズレません)
- 元の金額・送料を **点数按分**で元行と補助行に振り分けます(仕入れ総額は不変、両行の 1 点原価がほぼ同額で揃う)
- 修正後、元行に紐づく **登録済み商品の原価(仕入れ値)も自動で再同期**されます
- 完了画面に補助行の管理番号(例 **zA81~82**)が出ます。**残りの点数は、その補助行に対してスタッフアプリから登録**してください

点数を減らすとき(過大報告:例 52→50)

- 元行の点数・原価・管理番号(末尾を短縮)を修正し、登録済み商品の原価を再同期します
- 余った末尾番号(例 **zA51 ~**)は **未使用(欠番)**になります
- 次のいずれかに当てはまる場合は **自動で中止**され、警告が表示されます(手動対応が必要):
 - 余る末尾番号に **すでに商品が登録されている**(実点数より多く登録済み)
 - 対象行より後ろに **同区分の別バッチがある**(減らすと後続の番号がズれるため)

原価・登録済み商品はどうなるか

商品管理シートの「仕入れ値」は、商品登録時点の仕入れ管理「商品原価」を**コピーした値**で、通常は自動では直りません(`staff_apiCreateProduct` の 1 箇所だけが書き込み)。

この修正メニューは、点数修正で原価が変わると**紐づく登録済み商品の「仕入れ値・粗利・利益・利益率」を自動で再計算・上書き**します(販売前の商品も販売済みの商品も対象。計算式は `StaffApi.gs` の派生値計算と同一。シート式が入ったセルは尊重して上書きしません)。完了画面に再同期した件数が表示されます。

自動で中止されたとき(手動対応)

「点数を減らす」で後続バッチや登録済み商品があり中止された場合は、区分全体の振り直しが必要です。番号がズれても影響がない(区分内に作業済み商品がない)場合に限り、スプレッドシートを直接修正できます。

1. 「仕入れ数報告」シートで該当行の **G 列(処理済み)**を **FALSE**、**F 列(数量)**を正しい値に修正
2. onChange トリガーで `mergeReportToKanri_` が再マージし、`recalcAssignNumbers_` (`仕入れ数マージ.gs:351`) が区分全体を再採番



この直接修正は区分全体の管理番号を振り直します。**後続バッチに採寸・撮影・出品済みの商品が 1 つでもあるなら使わないでください。** 番号と画像(タスキ箱)が連動して壊れます。判断に迷う場合は無理に詰めず、余った番号を欠番のまま残してください。

改良時のマニュアル更新フロー

UI や機能を変更したら、以下の順序でマニュアルを更新します。

1. `/Users/katsu/saisun-repo/shiire-kanri/docs/manual/manual-staff.md` または `manual-admin.md` を編集
2. UI 変更が大きいなら関連スクリーンショットを撮り直して `screenshots/staff/` または `screenshots/admin/` に同名で上書き
3. `npm run build` で PDF 再生成
4. `dist/外注向けマニュアル.pdf` / `dist/管理者向けマニュアル.pdf` を関係者に共有

PDF ビルド詳細は `/Users/katsu/saisun-repo/shiire-kanri/docs/manual/README.md` を参照。

参考リンク

- GAS エディタ: スプレッドシート → 拡張機能 → Apps Script
- Cloudflare ダッシュボード: <https://dash.cloudflare.com/>
- Workers コンソール: <https://dash.cloudflare.com/?to=/:account/workers>
- CF Access: <https://one.dash.cloudflare.com/>
- BASE 管理画面: <https://admin.thebase.in/>
- 関連プロジェクトメモ: `/Users/katsu/.claude/projects/-Users-katsu/memory/`

付録: 各シートの初期化

新規スプレッドシートで運用を始める場合:

```
// GAS エディタで実行
inv_setupAllSheets()           // 請求書系 4 シート初期化
FULL_RESTORE_ALL()            // 全トリガー一括登録
```

すでに本番運用中の場合、これらの関数は冪等なので再実行しても安全です。

アップデート履歴

日付	版	変更内容
2026-06-21	v1.1	請求書に「追加報酬・控除」(手動明細レイヤー)機能を追加。スタッフ／管理者でレイヤーを分離(編集者は自レイヤーのみ操作、他レイヤーはサーバー側で保持)。請求書詳細モダルに管理者用エディタ・「自動算出を再計算(新番号で再発行)」ボタンを追加。全請求書タブに「振込用まとめCSV」出力を追加。PDF テンプレートが手動明細行を明細・小計・合計に反映。請求書履歴シートに「手動明細JSON」列を末尾追加。